

Component-1 Design Note — lexical & semantic retrieval on EB-NeRD and MIND

1. What was built

A config-driven pipeline from raw archives to metric reports in one command — `make data` for Q1’s feature store, `make all` for everything — across **EB-NeRD demo** (49k impressions / 1,590 users), **EB-NeRD small** (478k / 15,143) and **MIND-small** (230k / 50,000). Both datasets normalise into one schema, so every downstream stage has a single code path, not a Danish branch and an English one:

```
impressions_{train,val,test}.parquet  impression_id, user_id, timestamp, candidates[], clicked[],
                                       history[], history_timestamps[], recency_seconds
articles.parquet                      article_id, title, abstract, body, category, entities[],
                                       published_time, total_inviews, ...
```

`entities` normalises two source shapes — EB-NeRD’s `ner_clusters`, MIND’s per-field JSON — into one `List(String)` (74% / 87% non-empty); `recency_seconds` derives from `history_timestamps`, null where genuinely unknown. Nothing scores on either yet — the feature store is a contract, and a field added later is a migration.

Three scorers run over that schema — **bm25**, **emb**, **fused** — on two tracks: **re-rank** (score the pool the log actually showed; leaderboard-comparable) and **retrieval** (ignore the pool, pull the top-200 from the whole corpus — the “candidate generation to a few hundred” the spec asks for). The query for both is the user’s last 30 clicked titles. The harness reports AUC / MRR / nDCG@{5,10} with bootstrap CIs, diversity / novelty / coverage, cold-vs-warm and head-vs-tail slices, and paired comparisons.

2. Temporal split

Both datasets ship their own train/test boundary, so I adopted it as `test_start` rather than inventing one, and carved `val` out of the earlier block by time. EB-NeRD’s day boundary is **07:00**, not midnight — a detail that only shows up in the data.

	train	val	test
EB-NeRD small	168,522	64,365	244,647
MIND-small	95,071	61,894	73,152

EB-NeRD ships a separate `history.parquet` per block whose window ends exactly where its impressions begin, so the behaviour-window boundary is given rather than guessed. (The alternative — all 7 provided train days, halving the provided validation into val/test — buys ~38% more train data at the cost of test history up to 7 days stale; nothing here is trained, so fresh test history wins. NOTES.md, §splits.)

Enforcement, not discipline. `split.py` asserts on every build and fails rather than warns: strict `train < val < test`; per-item history-precedes-impression; `clicked` is a subset of `candidates`; every candidate resolves; no empty ids. Where a check is impossible it says so — MIND’s history has no per-item timestamps, so the harness prints that the guarantee is structural rather than verified, instead of passing vacuously.

`tests/test_leakage.py` (`make test`, 19 tests, ~20s) re-checks the same properties from *outside*, reading only `data/processed/`: the build asserts what it meant to write, the tests assert what is on disk. Two checks exist only there — that `datasets.yaml`’s cutoffs still match the artifacts, and that no module outside `eval/run.py` mentions a serving-unavailable column. I verified the suite is not vacuous by injecting a future click into a history and watching it fire.

3. Lexical axis

bm25s over title+abstract with a per-dataset Snowball stemmer. (**rank_bm25** scores one query at a time in pure Python — unusable at 65k documents; Pyserini/Lucene scales further but drags in a JVM.)

Stemming is not free, and not what I assumed. The brief specifies no preprocessing, so this was my choice. On val, `stem - none` is **+0.0019 [+0.0007, +0.0029] AUC on MIND, zero on EB-NeRD, and significantly negative on recall@200 for both** (-0.0020, -0.0023): it trades precision for variant-matching — worth it ordering ~8 candidates, harmful picking 200 from 20,738. Kept as right for the headline metric, but a track-dependent trade. Danish compound

splitting from corpus vocabulary fired on 15% of tokens and made both **worse**: compounding needs a lexicon, not statistics.

The finding that mattered. EB-NeRD first scored AUC **0.5035** — random. Rather than report that, I tested whether it was my bug: an **oracle test** (query = the clicked article’s own title) ranked it #1 in 300/300 impressions at AUC 0.9990 and a history-length sweep was flat, ruling out misalignment and query dilution. MIND worked at 0.58, and that asymmetry located the cause: **bm25s ships no Danish stopwords list**, so thirty concatenated Danish titles were mostly function words. The Snowball list moved EB-NeRD to **0.5232**. A missing stopwords list is silent — it degrades without erroring.

4. Semantic axis

MIND encodes title+abstract with **all-MiniLM-L6-v2 (11m16s)**, cached once). User vectors are mean of last-30 history vectors, L2-normalised; scoring is cosine with FAISS **IndexFlatIP** (exact at this scale; IVF/PQ deferred to 10× analysis). MIND’s corpus spans two files, so the index covers their 65,238-article union.

The encoder is a measurement, not a preference. Q3 allows the provided vectors *or* your own from BERT/XLM-RoBERTa, so I ran the identical pipeline on each — same splits, same BM25, same fusion, only the article vectors differ — selecting on **val**, since selecting on test tunes a choice on the number being reported. Semantic-only AUC, **ebnerd_small val**, against BM25’s 0.5205:

encoder	dim	val AUC	
paraphrase-multilingual-mpnet-base-v2 (ours, XLM-R)	768	0.5309 [0.5282, 0.5336]	beats BM25
Ekstra_Bladet_word2vec (provided)	300	0.5098 [0.5073, 0.5124]	loses
google_bert_base_multilingual_cased (provided)	768	0.4857 [0.4832, 0.4880]	below random

One property predicts all three: **was the model trained so that cosine distance means topical similarity?** XLM-R fine-tuned on paraphrase pairs was; word2vec (predict neighbouring words) was not; raw mBERT (fill in blanks) emphatically was not — its vectors sit in a narrow cone where everything looks alike. Size predicts nothing: the 300-d model beats the 768-d one. Fusion detects the failure unprompted, tuning alpha to 0.00 on demo — discarding the semantic axis outright.

MIND kept MiniLM, and that is the sharper lesson. Against the same XLM-R, MiniLM wins on **val** (0.6338 [0.6312, 0.6362] vs 0.6256 [0.6232, 0.6279]) and *loses* on **test** (0.6339 vs 0.6364) — a real reversal, near-disjoint intervals. The rule was fixed before looking, so MiniLM stays; “take the better test number” would have flipped it. Obeying the rule cost 0.0025 test AUC, which is the entire point of having one. The direction is sensible too: an English specialist beats a 50-language generalist on English, while that generalist beats a 2013 word-vector model on Danish.

5. Candidate generation: recall@K, and where the winner changes

Recall@K over the **full catalogue** — not the pool the log showed — is what measures candidate generation: the share of an impression’s clicked articles found in the top K. Cold-start impressions retrieve nothing and score 0 rather than being excluded — a retriever that cannot serve a user has failed for that user.

test, recall@K	K=50	K=100	K=200	emb — bm25 @200
EB-NeRD small · bm25	0.0072	0.0133	0.0247	
EB-NeRD small · emb	0.0043	0.0081	0.0150	−0.0097 [−0.0105, −0.0089]
MIND-small · bm25	0.0060	0.0124	0.0220	
MIND-small · emb	0.0076	0.0138	0.0239	+0.0019 [+0.0008, +0.0031]

Which retriever is better has no dataset-wide answer. BM25 wins on EB-NeRD at every K, embeddings win on MIND, both significantly — same code, same K, opposite verdicts, so a result quoted from one dataset does not transfer.

Retrieving and re-ranking are different skills, and the same encoder does one well and the other badly. On EB-NeRD the XLM-R vectors *re-rank* significantly better than BM25 (+0.0198 AUC, §6) while *retrieving* 40% worse (0.0150 vs 0.0247 @200) — starkest on popular articles, where they re-rank at AUC 0.6801 [0.6597, 0.6995] against 0.5852 [0.5626, 0.6066] but still retrieve them worse. Ordering ~8 candidates the log already filtered and finding 200 in a 20,738-article catalogue are different jobs, and nothing says one signal must win both — the argument for different signals per stage. Absolute recall is low everywhere (2–4%), see §8.

6. Fusion — an addition the brief did not ask for

Q3 asks only to *compare* lexical and semantic; this third system is mine. BM25 scores and cosines are on different scales, and BM25’s moves with query length, so both are z-normalised **within each candidate pool** and mixed $\alpha \cdot \text{emb} + (1 - \alpha) \cdot \text{bm25}$, α tuned on **val**, applied unchanged to test.

test, AUC	bm25	emb	fused	alpha	fused – emb
EB-NeRD demo	0.5125	0.5295	0.5300	0.65	+0.0005 [−0.0013, +0.0026] — includes zero
EB-NeRD small	0.5107	0.5305	0.5291	0.60	−0.0014 [−0.0022, −0.0007] — excludes zero
MIND-small	0.5645	0.6339	0.6353	0.80	+0.0014 [+0.0005, +0.0023] — excludes zero

It mostly does not pay, and that is the useful part. On `ebnerd_small` fusion wins on val (0.5376 vs 0.5309) and is *significantly worse* on test — the alpha did not generalise, so **the reported EB-NeRD system is embeddings alone**, and only MIND ships fused, for +0.0014. The harness is what caught it: on val alone fusion looked a clear win on both datasets. The same instability appeared earlier under a different encoder, so EB-NeRD’s optimal mix genuinely drifts rather than this being a one-off.

7. Evaluation harness

Metrics are computed **per impression**, which makes everything else possible: slicing is a mask over that vector, bootstrapping resamples it. Resampling is over impressions, the approximately-independent unit — resampling candidates within one would badly understate variance. Comparisons use a **paired** bootstrap on shared resamples, since two systems on the same impressions have correlated errors and comparing two independent intervals is far too conservative.

Cold-start needed a defensible definition. “Empty history” fails on EB-NeRD, which has *zero* such users (median history 258 clicks) — it would select an empty slice and silently report nothing. The harness uses a per-dataset bottom-decile history-length threshold plus the true zero-history slice where one exists. On MIND that slice scores 0.5125 identically across all three systems, correctly: with no history there is no query, so no ranking is invented.

Beyond-accuracy over the top-10: category intra-list diversity, novelty as self-information against *train* click shares, and coverage — sobering, at **5–21%** of the catalogue ever surfaced.

8. Observations

Content signals are weak here, and that is the result, not a failure. MIND reaches 0.6353, within reach of published neural baselines for MIND-small (~0.65–0.67) from an entirely unsupervised system. EB-NeRD tops out at 0.5305, for a structural reason: its pools are much smaller (median 8–9 vs MIND’s 23–26) and drawn from a tight recency window, so every candidate is already plausible and topically close — there is little for content to separate.

Simple behavioural features are not the shortcut they look like. Train-click popularity scores *below* random on EB-NeRD (0.459) — the pool is already popularity-filtered.

Demo was representative. Every system landed within 0.002 of its demo value at 10× the users; the extra data bought precision — CIs tightened from ± 0.0040 to ± 0.0013 , which is what decided fusion.

The retrieval track is weak regardless of scorer (recall@200 of 2–5%): EB-NeRD’s corpus spans 1998–2023 while impressions are one week in May 2023, and `published_time <= impression_time` removes only *future* articles, not stale ones. A recency window on the retrievable corpus is the obvious lever, and belongs with the behavioural signals in C2.

9. Anti-gaming and serving availability

`total_inviews/total_pageviews/total_read_time` are lifetime aggregates over the whole collection period — they embed the future and cannot be computed at serving time. They are carried into the processed corpus **only** so the harness can quantify them; no shipped scorer reads them.

test, AUC	without	with popularity	difference
EB-NeRD demo	0.5300	0.5728	+0.0427 [+0.0401, +0.0455]
EB-NeRD small	0.5291	0.5731	+0.0440 [+0.0431, +0.0449]

One serving-unavailable feature is worth **+0.044** — more than twice the entire semantic axis (**+0.0198, §6**), which was the largest legitimate win in this project, and the single biggest effect anywhere in the report. Any comparison that quietly includes it is not measuring the same system, and any leaderboard it wins is measuring hindsight rather than recommendation.

Two further leakage vectors are enforced, not trusted: the user’s *next* action and post-click engagement (`next_read_time`, `next_scroll_percentage`, `read_time`, `scroll_percentage`) are dropped at the split, and retrieval filters `published_time <= impression_time` — 867 EB-NeRD articles postdate the test window.

10. Where it breaks at 10× — measured, not projected

The leaderboard submission forced this section to stop being hypothetical. Codabench scores **ebnerd_testset: 13.5M impressions, 205,925,868 candidate pairs — 28× ebnerd_small**.

Memory is the wall, and it is one specific idiom. Three separate times the thing that broke was `.to_list()` pulling Arrow into boxed Python objects: `np.vstack(col.to_list())` peaks at **5.8 GB to build a 385 MB array**, and the same idiom OOM-killed the harness at 10 GB RSS while adding `recall@K`. `explode().to_numpy().reshape(...)` plus Polars set intersections fixed all three, byte-identically.

Streaming works and costs little. `pipeline/submit.py` scores in 200k-impression slices: peak RSS on the full 13.5M-impression run was **5,912,172 kB against a 20k smoke test’s 5,912,100 kB** — flat in dataset size to within 0.001%, not linear — in 33m28s; MIND-large’s 2.4M took 6m05s at 1.3 GB. Polars slice pushdown reaches the parquet row groups, so a slice at offset 13M costs the same as one at 0 (0.05s either way); without it streaming would be $O(n^2)$. MIND needed a one-off TSV→parquet conversion first — CSV has no such pushdown, and reading it whole (3.07 GB to parse) OOM-killed the first attempt.

BM25 is the piece that genuinely does not scale, and I found this by trying. `get_scores` returns a dense score over the entire corpus per distinct query; on MIND-large test that is ~2M distinct histories × 120,961 articles, on the order of 100 billion operations — hence embeddings-only submissions (§11). The conclusion is the real one: a dense scorer cannot back a two-stage system at this size, and an inverted index with early termination (WAND) stops being optional. Projected, not measured, at **ebnerd_large** (600M): FAISS flat is $O(N \cdot d)$ per query and would need IVF-PQ with `nlist` ~ $\sqrt{N}$.

11. Leaderboard submissions

Both competitions score held-out sets far larger than the splits above — **ebnerd_testset** (13.5M impressions) and MIND-large test (2.4M) — so `pipeline/submit.py` streams them (§10) with unchanged scoring semantics. **Both entries are embeddings-only rather than fused**, for the BM25 scaling reason in §10: free on EB-NeRD, where embeddings alone *is* the reported system (§6), and a known 0.0014 AUC on MIND, reported as a different system rather than letting the stronger fused number stand in for the one uploaded. EB-NeRD allows five submissions a day and scores for hours, so output is validated offline, not by trial: all 13,536,710 lines checked as rank permutations of the right length in the test file’s exact row order, plus a sample re-ranked against an independent recomputation. **Leaderboard screenshots:** *(insert after uploading — MIND 13967, RecSys 2024 2469)*